Real-time CTR prediction with online feature selection - Project report

Hubert Jaczyński, Aleksandra Kłos, Jakub Oganowski

8th June 2026

1 Introduction

1.1 Motivation

The modern digital advertising market is based on real-time bidding (RTB)[1]. In this type of system, the final impression value, which is measured by the eCPM metric[2], is dependent on the estimation that the user will interact with a given advertisement. Therefore, the precise estimation of click-through rate (CTR)[3] serves as an economic basis of the whole industry. In 2013, Google engineers wrote a paper presenting a selection of case studies and topics drawn from recent experiments in the setting of a deployed CTR prediction system and describing the architecture of the FTRL-Proximal model [1]. Moreover, they have pointed out that, on a global scale, even a minor improvement in prediction accuracy results in multimillion-dollar savings and a significant increase in content relevance for end users.

Nevertheless, this challenge is rather complex from both computational and analytical viewpoints. This is because RTB systems are obliged to process hundreds of thousands of queries per second under very rigorous time constraints. Moreover, the incoming ad logs have the character of an endless stream and are characterized by two problematic features: significant sparsity, as the vast majority of impressions do not end with a click, as well as considerably high dimensionality of data [2]. It stems from the fact that most signals are categorical variables with millions of unique values, such as unique IP addresses, device identifiers, and so forth.

Common machine learning methods using batch streaming are impractical in such circumstances. On the other hand, constant retraining of powerful models from scratch using a growing database is too expensive in terms of time and infrastructure. This requires the use of incremental learning algorithms. They can update their weights and structure on the fly, processing data instance by instance, immediately after receiving a notification of a click or its absence.

[1] RTB House, *Real-time bidding*, `https://www.rtbhouse.com/martech-terminology/real-time-bidding`.

[2] Unity, *What does eCPM stand for?*, `https://unity.com/glossary/ecpm`.

[3] Google Ads, *Clickthrough rate (CTR): Definition*, `https://support.google.com/google-ads/answer/2615875?hl=en`.

1.2 Research problem

The main issue in analyzing advertising data streams is their inherent mobility. Machine learning models are trained under the assumption of a constant data distribution, but in reality, this distribution changes over time. It is known as concept drift [3]. This change can affect both user preferences, such as seasonal purchasing patterns, and the effectiveness of advertising itself.

Besides, the phenomenon of feature drift, described in the literature on online feature selection [4], is especially applicable. It means that the set of features relevant to the model changes over time. Some parameters, for instance, the user's phone model, may suddenly become more important or become irrelevant altogether. As presented in the lecture, algorithms, such as decision trees, often fail to keep pace with these changes. Thus, they bring about the adaptation gap.

1.3 Research objective

Based on the observations described, our project aims to develop a state-of-the-art model capable of rapid adaptation in a non-stationary environment. The streaming random patches (SRP) algorithm [5] has been chosen to be the foundation. We have decided to choose it as it is currently one of the most effective ensemble solutions for data streams. Furthermore, the literature states that SRP is characterized by the use of background models that begin learning a new concept immediately after the ADWIN detector detects a drift.

Yet, our project's innovation extends the SRP mechanism with dynamic feature selection, described as online feature selection (OFS) [6]. Our algorithm still initialises the ensemble with random feature subspaces, as in standard SRP, but instead of leaving these subspaces fixed it evaluates each feature's usefulness on the fly and repairs the affected subspaces when drift is detected. This may show that when drift is detected, the reinitialised ensemble members receive access to the most recent and informative features. They eventually focus on shortening recovery time and reducing logarithmic loss [7].

2 Data preparation

For this project, we have prepared two real advertising streams and five controlled synthetic stream configurations. The real streams, Avazu and Criteo, evaluate the method on high-dimensional CTR data. The synthetic configurations allow us to test abrupt drift, gradual drift, label/function noise, irrelevant features, and high-dimensional feature spaces under known drift locations.

2.1 Real-time data streams: Avazu and Criteo

RTB datasets are characterized by their vast volume and sparsity. Special care was taken to preserve the original chronological order of the rows. This is because random data shuffling could destroy the naturally occurring concept drift. Hence, the evaluation becomes impractical.

2.1.1 Exploratory data analysis – Avazu dataset

The dataset has been obtained from the Kaggle competition[4]. The competition data provides separate training and testing files. However, for the purpose of this EDA and later for model evaluations, we have used `train.csv` data. Besides, to save RAM on our devices, while preserving the nature of streaming data, the first $1,000,000$ records has been analyzed.

The subset consists of 24 columns in total: 23 predictive features and 1 target variable (`click`). The vast majority of the features are categorical and include explicit attributes describing the user's hardware and site context, such as `device_id`, `device_ip`, `site_domain`, `app_category`, as well as anonymized categorical variables like `C1`, `C14`–`C21`.

Target variable and class imbalance The target feature, `click`, is a binary variable that indicates whether an advertisement was clicked (1) or not (0). The distribution analysis showed a serious class imbalance. It is a common characteristic of RTB systems. Specifically, only 16.02% of the impressions resulted in a click, whereas 83.98% were ignored. As seen in Figure 1, this highly skewed distribution ($P(Y = 0) \gg P(Y = 1)$) disqualifies the use of the traditional accuracy metric and encourages us to use the probabilistic logarithmic loss function for further model evaluation.

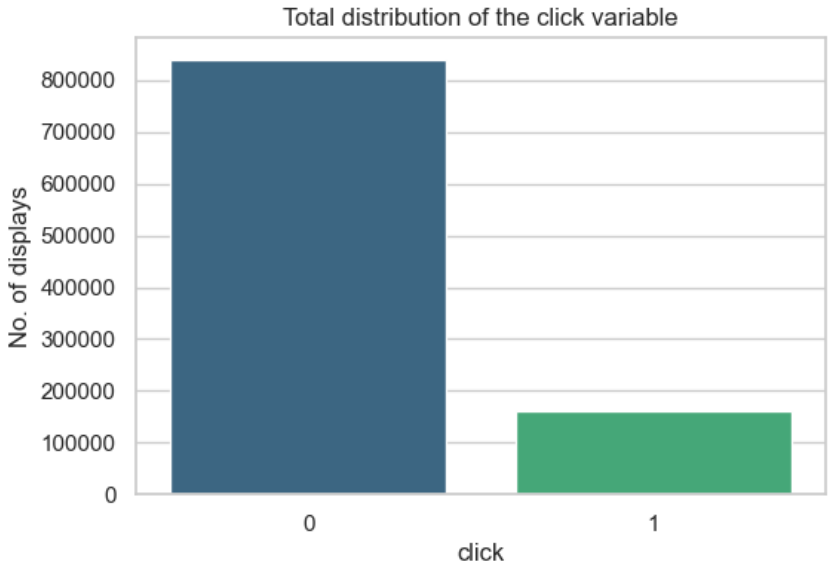


Figure 1: Avazu dataset: Total distribution of the `click` variable.

High cardinality of categorical features The major challenge of the Avazu dataset is the high cardinality of its categorical features. Excluding the unique `id` column, the analysis showed that single features contain up to hundreds of thousands of unique values. As seen in Table 1, within the $1,000,000$ rows sample, the `device_ip` column contains $313,002$ unique values, `device_id` contains $83,431$, and `device_model` contains $4,581$ unique categories. Such dimensionality makes one-hot encoding computationally infeasible, which justifies the use of hashing techniques.

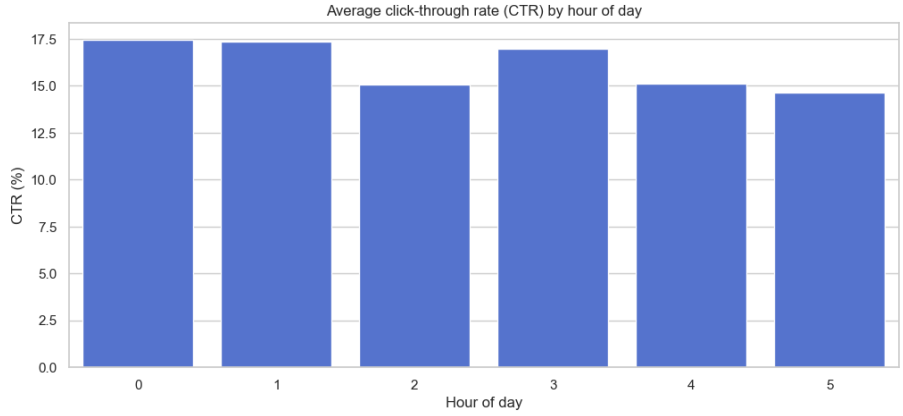
[4]Kaggle, *Click-Through Rate Prediction - Avazu*, `https://www.kaggle.com/competitions/avazu-ctr-prediction/`.

Table 1: Avazu dataset: Feature cardinality of the first 5 features.

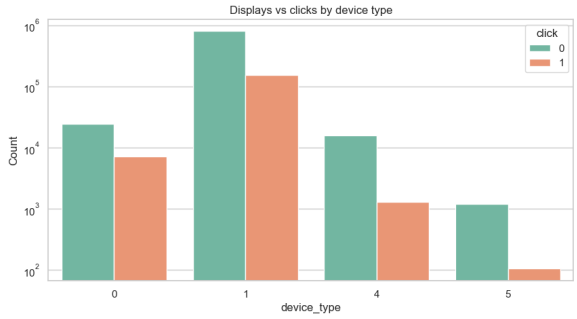
Feature name	No. of unique values
`id`	1,000,000
`device_ip`	313,002
`device_id`	83,431
`device_model`	4,581
`app_id`	2,309

CTR analysis Further analysis of the CTR has been conducted across different dimensions. The analysis made was as follows, as provided in Figure 2:

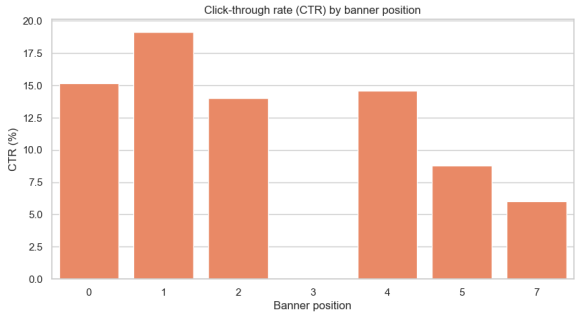
- **Temporal variability (plot A):** The average CTR is not constant but fluctuates depending on the hour of the day. It suggests the presence of concept drift in the stream.
- **Device type influence (plot B):** The distribution of displays and clicks varies by `device_type`. Even though one device dominates overall traffic, click rates vary by platform.
- **Advertisement placement (plot C):** The analysis of the `banner_pos` feature demonstrated a correlation between the physical placement of the banner on the screen and the resulting CTR.



(a) Avazu dataset: Average CTR by hour of day



(b) Avazu dataset: Displays vs clicks by device type



(c) Avazu dataset: CTR by banner position

Figure 2: Avazu dataset: CTR analysis.

2.1.2 Exploratory data analysis – Criteo dataset

The Criteo dataset is an online advertising dataset released by Criteo Labs[5]. It contains feature values and click feedback for millions of display advertisements and serves as a widely used benchmark for CTR prediction. As in the Avazu dataset, we used the `train.txt` file, and $1,000,000$ records were analyzed.

The dataset is heterogeneous and comprises 40 columns. The first attribute is the target variable (`label`), where 1 indicates the advertisement was clicked and 0 indicates it was not. The remaining 39 predictive attributes consist of 13 continuous numerical columns (`I1`–`I13`) and 26 anonymized categorical columns (`C1`–`C26`).

Target variable and class imbalance The target feature, `label`, is a binary variable representing user interactions. Once again, as presented in Figure 3, the distribution analysis of the sampled records revealed a notable class imbalance. Particularly, 25.49% of the impressions resulted in a click, while 74.51% were ignored. Although the CTR is slightly higher compared to the Avazu dataset, the data remains heavily skewed ($P(Y = 0) > P(Y = 1)$) and we cannot use the traditional accuracy metrics.

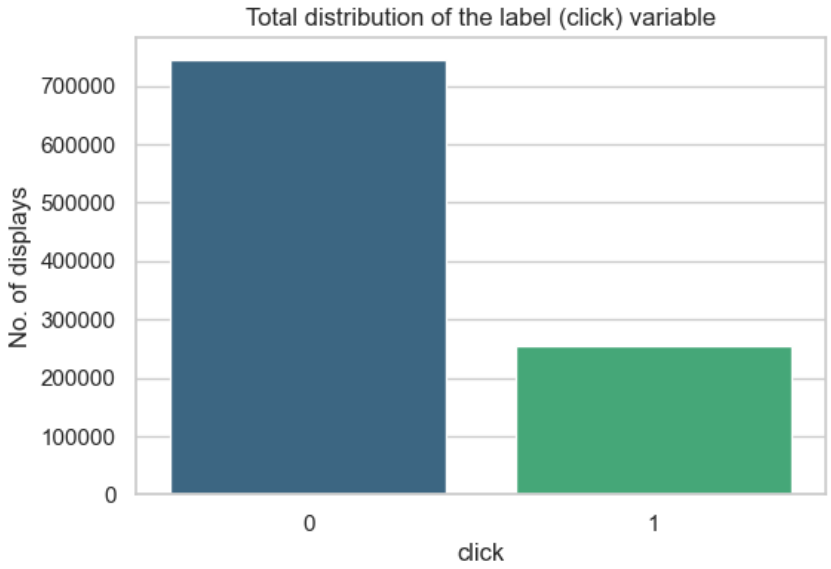


Figure 3: Criteo dataset: Total distribution of the `label` (click) variable.

Missing data and numerical skewness Unlike the Avazu dataset, the Criteo dataset poses the challenge of continuous variables. EDA showed that the numerical features (`I1`–`I13`) are sparsely populated, with varying percentages of missing values (NaNs) across columns. Furthermore, the existing continuous values exhibit highly right-skewed, long-tail distributions, as shown in Figure 4. They are populated by values close to zero but contain vivid outliers, as shown by truncating feature `I1` at the 99th percentile. The sparseness and skewness have required data preprocessing and an imputation strategy, such as filling missing numerical values with zeros, before model training.

High cardinality of categorical features The 26 categorical features (`C1`–`C26`) consist of 32-bit hashed strings. They exhibit a high cardinality similar to the Avazu dataset,

[5]Criteo AI Lab, *Criteo Research Datasets*, `https://ailab.criteo.com/ressources/`

despite the fact they have remained anonymized.

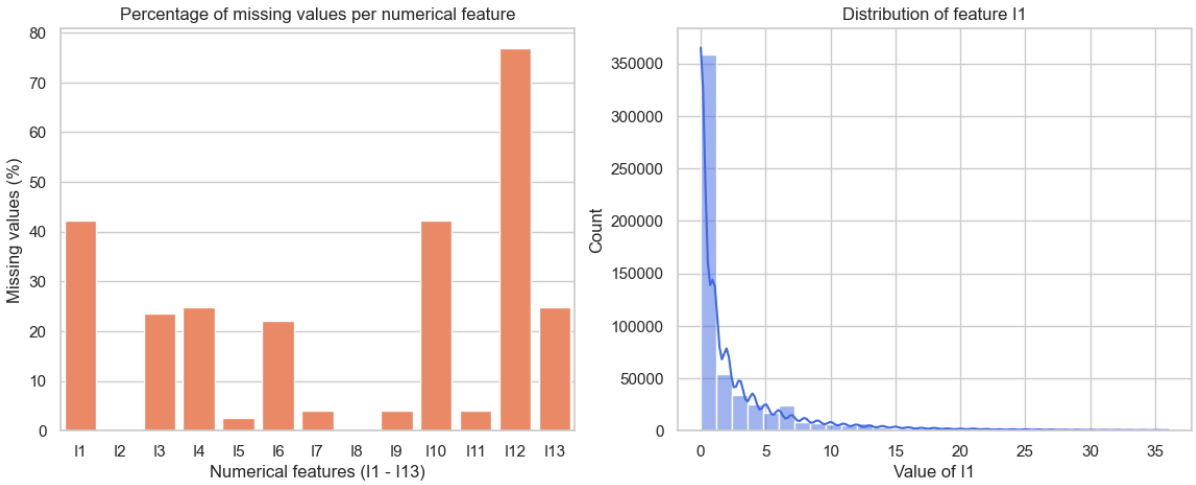


Figure 4: Criteo dataset: (A) Percentage of missing values per numerical feature, (B) Skewed distribution of continuous feature `I1`.

2.2 Feature engineering and hashing

As demonstrated in EDA, data streams with high cardinality and missing values can easily overload an online classifier's memory. To solve this, we have built a code that creates new features, reduces dimensionality, and saves the final output as an `ARFF` file for the Massive Online Analysis (MOA) framework [8].

Temporal and moving window features To help the model adapt to recent trends and concept drift, we have created dynamic features:

- **Avazu:** We have extracted specific time details (hour, day, week) and created interaction features such as `site_x_hour`. This feature is built as a raw interaction string of the form `site_id` + `"_h"` + hour-of-day; together with the other categorical columns it is then passed to the feature hasher, so the interaction is hashed afterwards rather than being a concatenation of already-hashed values. We have also used a sliding window of 1000 instances to calculate the recent site CTR, the recent app CTR, the recent site frequency, and the site frequency change.
- **Criteo:** The moving averages, standard deviations, and deviations from the mean were first computed on the raw numerical columns (`I1`–`I5`) using a sliding window of 1000 instances, with missing values skipped by the windowed estimator (`min_periods=1`). Only after these statistics have been recorded, the remaining `NaN` values in the original columns have been replaced by zero. This order ensures that zero-imputed entries do not distort the window statistics.

Dimensionality reduction via feature hashing As mentioned in Section 2.1.1, techniques like one-hot encoding create too many columns for streaming data. Instead, we have decided to use the feature hashing algorithm [9] from *scikit-learn* library to compress all categorical strings into a fixed-size vector of exactly 100 columns. This has provided two main benefits:

1. The model's size stays the same, even if new, unseen categories appear in the stream.
2. By keeping the number of columns strictly bounded, the SRP ensemble can process data much more quickly.

Final state In the final step, we have merged all these parts together. For Avazu, we have combined the new time-based features with the 100 hashed columns. For Criteo on the other hand, we have combined the 13 original numeric columns, the 15 new moving statistics, and the 100 hashed columns. This optimized data has been then to train and evaluate our models.

2.3 Synthetic data streams

Real-world streams contain natural, hidden concept drifts, but controlled synthetic streams are necessary to verify whether an adaptation mechanism reacts correctly when the drift location and drift type are known. In response to the reviewer comment, the final evaluation was extended from the original Agrawal-only synthetic setup to five synthetic stream configurations. Every synthetic run contains 200,000 instances and places the main drift at instance 100,000.

Agrawal variants The Agrawal generator [10] simulates a loan-approval process using 9 predictive attributes. We use three variants: `agrawal_sudden`, where the classification function switches abruptly from function 1 to function 3; `agrawal_gradual`, where the same concept change is spread over a 40,000-instance transition window; and `agrawal_noisy`, where a sudden function change is combined with 10% perturbation noise. These variants test whether the models remain stable under abrupt, gradual, and noisy drift.

LED irrelevant-attribute stream The `led_irrelevant` stream is based on MOA's LED generator with 7 relevant digit-segment attributes and 17 irrelevant attributes. Before the drift, 3 relevant attributes are swapped with irrelevant ones; after the drift, 7 attributes are swapped. This stream directly tests whether feature-selection mechanisms can cope with a changing set of relevant and irrelevant attributes. Since LED is a 10-class stream, the AUC values reported for this dataset should be interpreted cautiously; the main metrics used for comparison are Accuracy, Log Loss, and windowed Log Loss.

High-dimensional RBF stream The `rbf_highdim` stream uses two MOA Random RBF concepts with 100 numeric attributes, 2 classes, and 50 centroids. The stream switches from seed 1 to seed 2 at the drift point. This satisfies the reviewer's request for a larger-dimensional synthetic stream and tests whether the proposed method behaves sensibly when the feature space is closer to the dimensionality of the real hashed CTR streams.

2.4 Summary of data streams characteristics

Following the EDA, feature engineering, dimensionality reduction, and synthetic-stream extension phases, seven data streams were evaluated sequentially in the Java/MOA

pipeline. Table 2 presents the final characteristics of the streams used in the 200,000-instance final evaluation.

Table 2: Summary of the final data streams used for model evaluation.

Dataset	Instances per run	Final feature types	Feature count	Concept drift type
Avazu	200,000	Numeric + hashed categorical	109	Natural CTR drift
Criteo	200,000	Continuous + hashed categorical	128	Natural CTR drift
Agrawal sudden	200,000	Mixed synthetic	9	Abrupt function drift at 100,000
Agrawal gradual	200,000	Mixed synthetic	9	Gradual function drift centered at 100,000
Agrawal noisy	200,000	Mixed synthetic + noise	9	Abrupt function drift with 10% perturbation noise
LED irrelevant	200,000	7 relevant + 17 irrelevant binary attributes	24	Relevant/irrelevant attribute swap at 100,000
RBF highdim	200,000	Numeric RBF features	100	Abrupt 100-dimensional concept switch at 100,000

3 Evaluation methodology

3.1 Drift detection in real-time stream

First, we wanted to verify the presence of concept drift in the Avazu and Criteo datasets. To investigate it, we have used the ADWIN (ADaptive WINdowing) algorithm [11, 12], implemented via the Python `river` library. It expands the window when the data are stationary and abruptly shrinks it when a statistically significant change in the stream's mean is detected.

Experiments We have processed the chronological sequences of both the Avazu and Criteo streams, using the ADWIN estimator to continuously monitor the moving average of the target variable. The CTR and the detected drift points are presented in Figure 5 and Figure 6.

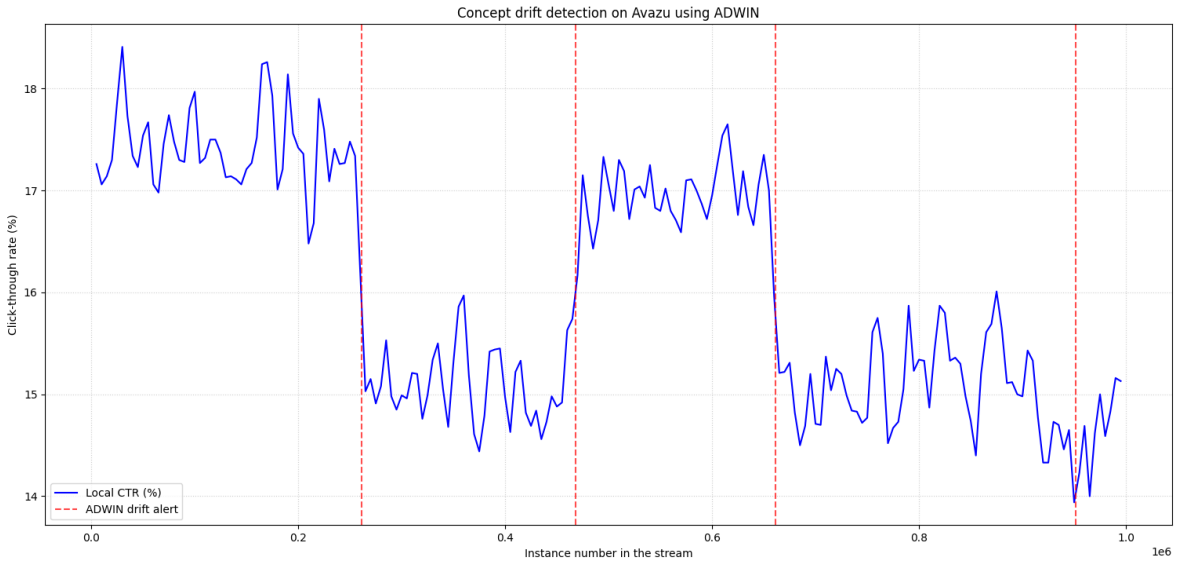


Figure 5: Avazu dataset: Concept drift detection using the ADWIN algorithm.

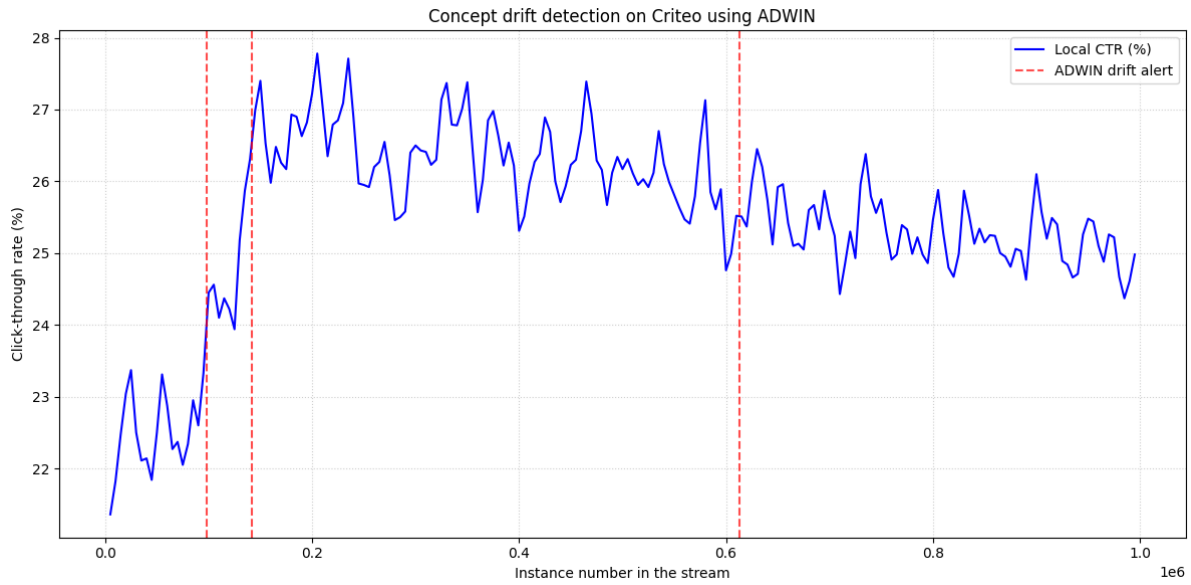


Figure 6: Criteo dataset: Concept drift detection using the ADWIN algorithm.

As shown in the above plots, the CTR is highly volatile and not constant over time. For the Avazu stream, the local CTR fluctuates between 14% and 18.5%, triggering four different drift alerts over $1,000,000$ points. Similarly, the Criteo stream shows non-stationarity. CTR climbs from initial values around 21% up to almost 28%, setting off three main drift alerts.

As we can see, the ADWIN detector has identified critical points in both datasets where the probability distribution underwent major shifts. Hence, the presence of concept drift in these streams requires us to use dynamic online learning models to maintain predictive performance.

It is crucial to note that monitoring the CTR (target variable) is not the only way to detect drift in these streams. Concept drift can also manifest itself as changes in the marginal distributions of individual input features (feature drift) or a deterioration in the prediction error rate of the trained model (error rate drift). Relying solely on the target variable can therefore allow us to miss drifts affecting feature importance without immediately changing the class ratio. Extended drift detection experiments based on model error rates and feature distributions are presented separately in the Section 5.

3.2 Evaluation procedure in the MOA environment

Due to the continuous nature of data streams, traditional machine learning evaluation techniques, such as cross-validation, are not applicable. We have used the prequential evaluation (test-then-train) method. In this approach, each incoming instance is first used to test the model – generating a prediction – and is then passed to the training algorithm to update the model's internal state. This means the classifier is always evaluated on unseen data.

Both real-world streams show class imbalance. Hence, we have abandoned the standard accuracy metric, which favors the majority class. Instead, we decided to evaluate our models using logarithmic loss (Log Loss) and area under the ROC curve (AUC ROC) [13]. The logarithmic loss heavily penalizes confident yet incorrect predictions, whereas

the AUC-ROC evaluates the model's ability to correctly separate the positive (clicked) and negative (non-clicked) classes.

3.3 Proof-of-concept baseline: HT in MOA

We first established a proof-of-concept baseline using the Hoeffding Tree (HT) – a basic incremental decision tree for high-speed data streams. The HT model was selected as the initial reference point because it is the simplest streaming tree learner: it grows splits incrementally but has no built-in drift-handling mechanism, which makes it a natural non-adaptive baseline against which adaptive methods can be compared. The baseline HT model has been evaluated on our processed Avazu and Criteo streams using the prequential procedure. This preliminary evaluation was conducted in the MOA graphical interface and served to confirm that the data preprocessing produced sensible results before proceeding to the Java-based experiments described in Section 4 and Section 5.

3.4 Adaptation gap on the synthetic stream

We have evaluated a HT baseline on the synthetic Agrawal stream. We have injected an abrupt concept drift (a switch from Agrawal classification function 1 to function 3) exactly at instance 100,000, consistent with the synthetic stream configuration used in the final evaluation. Figure 7 illustrates the resulting adaptation gap.

Figure 7: Agrawal dataset: Error of the baseline HT model over time, showing the adaptation gap after an abrupt concept drift. Sudden drift takes place at instance 100,000.

During the initial stable phase, from 0 to 100,000 instances, the HT has successfully learned the defined rules and has driven its classification error down to roughly 6%. However, immediately upon drift injection, the previously learned tree splits became obsolete, causing the classification error to rise to a peak of about 10%. Because a plain Hoeffding Tree cannot discard outdated splits, it can only recover by gradually growing new structure on top of the stale one, so the error decreases slowly back towards 8% by the end of the stream.

The resulting rise-and-recovery forms a distinct triangular shape on the plot, whose width represents the recovery time ($t_{recovery}$). This leaves us with an important observation. Namely, standard non-adaptive tree learning recovers from drift relatively slowly. Therefore, flattening this exact error curve and minimizing $t_{recovery}$ are our primary motives for investigating the dynamic OFS as the next milestone.

4 Architecture of the proposed solution

4.1 Overall streaming pipeline

The implementation in the repository follows the standard streaming-learning loop: each instance is read in chronological order, transformed into a fixed feature representation, predicted by the current model, scored, and only then used for training. This protocol — known as prequential or test-then-train evaluation [3] — prevents information leakage from future observations into the current prediction and provides an unbiased estimate of the model's behaviour as a function of stream position.

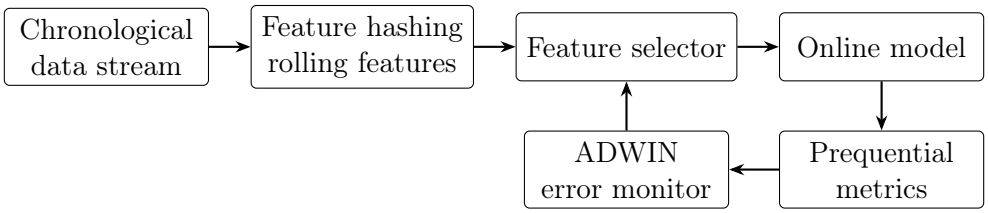


Figure 8: Architecture of the streaming evaluation and adaptation pipeline.

The Java implementation is organised around five groups of classes. Stream providers load either ARFF-based real datasets or the Agrawal, LED, and Random RBF synthetic streams through a unified `StreamProvider` interface. Model wrappers expose a common `StreamModel` contract for Hoeffding Tree, Hoeffding Adaptive Tree, SRP, and the proposed drift-aware SRP model. Feature selectors implement no selection, static top-K selection, periodic online ranking, and the drift-aware selection introduced in this work. The prequential evaluator coordinates prediction, metric updates, drift detection, model training, and CSV logging in a single integration loop. Decoupling the components through interfaces makes it possible to evaluate any combination of provider, selector and model without modifying the evaluator itself.

Algorithm 1: Main prequential evaluation and adaptation pipeline

Input: Chronological stream S, feature selector F, stream model M, drift detector D, metrics $\mathcal{M}$

Output: Metric history, final metrics, detected drift events

Initialise F on the stream header;
Initialise M on the filtered header returned by F;
Reset all metrics and reset D;
foreach *incoming instance* (x_t, y_t) *in chronological order* **do**
    $\tilde{x}_t \leftarrow F.\text{filter}(x_t)$;
    $\hat{p}_t \leftarrow M.\text{predictProbability}(\tilde{x}_t)$;
    Update every metric in $\mathcal{M}$ using $(\hat{p}_t, y_t)$;
    $e_t \leftarrow \mathbb{1}[\mathbb{1}(\hat{p}_t \geq 0.5) \neq y_t]$;
    if $D.\text{detect}(e_t)$ **then**
        Store a drift event at time t;
        Notify the drift handler attached to F and/or M;
    Train M on $(\tilde{x}_t, y_t)$;
    Update the feature selector buffer with the raw instance (x_t, y_t);
    if t *is a logging point* **then**
        Store the current metric snapshot;

4.2 Baseline models and feature selectors

The baseline model set contains Hoeffding Tree (HT), Hoeffding Adaptive Tree (HAT), and Streaming Random Patches (SRP). HT is the basic incremental tree model for high-speed streams [14] and serves as a non-adaptive reference. HAT extends the tree with ADWIN-based monitors at internal nodes and replaces obsolete subtrees with alternative subtrees as the underlying concept changes. SRP is an ensemble method designed for evolving streams [5]; it combines online bagging with random feature subspaces and per-learner drift detection. In the repository, the SRP baseline uses 10 trees and a 60% random subspace setting, consistent with the default values used by the original authors.

Three baseline feature-selection strategies were implemented and evaluated. In all selectors, Information Gain is computed on the selector's current sample buffer, not on the full future stream. Static top-K uses the initial 5,000-instance warm-up buffer; online ranking uses the most recent 5,000-instance sliding buffer; drift-aware selection uses one pre-drift and one post-drift buffer. For each feature X_i, the ranker computes $IG(X_i; Y) = H(Y) - H(Y \mid X_i)$ with respect to the observed class attribute Y. Nominal attributes use empirical class counts per value, while numeric attributes are discretised into 10 data-dependent bins before conditional entropy is estimated.

- **No selector**: all attributes are passed to the model. This is the natural reference for measuring whether feature selection helps at all on a given stream.
- **Static top-K**: Information Gain is estimated on a warm-up segment of 5,000 instances and the selected features remain fixed for the rest of the stream.
- **Online ranking**: Information Gain is recomputed on a sliding window of 5,000 instances every 5,000 instances. The selected feature set therefore evolves with the stream, but only on a fixed schedule.

These baselines isolate the contribution of dynamic feature selection from the contribution of the classifier itself, and provide a comparison point for the proposed drift-aware variants.

Feature selection influences all non-DASRP models through the instance header passed to the classifier. The selector physically filters the incoming instance to the currently selected attributes and appends the class attribute at the end. Therefore HT, HAT, and vanilla SRP never receive attributes outside the selected subset. When a dynamic selector changes the subset, the affected monolithic model is reinitialised on the new filtered header. Past tree splits are not retroactively constrained to the newly selected features; instead, the old model state is dropped and a new model starts learning from the new feature space. Static top-K does not trigger later resets because its selected subset is fixed after warm-up.

4.3 Drift-aware feature selection

The proposed drift-aware selector replaces the periodic schedule of the online ranking baseline with an event-driven adaptation triggered by ADWIN. The key observation is that the meaningful signal for feature selection is not the absolute Information Gain at a single point in time, but rather the change in Information Gain caused by the drift itself: features whose predictive value has dropped should be replaced precisely by features whose predictive value has risen.

The selector maintains a sliding buffer of recent instances. While no drift is signalled, the buffer simply rolls forward. When ADWIN raises a drift alert, the selector freezes the current buffer as the pre-drift snapshot and records the corresponding Information Gain values IG_{before}. It then continues to consume instances normally for one further window, accumulating post-drift data. Once the post-drift window is full, IG_{after} is computed and the per-feature change is taken as

$$\Delta IG_i \;=\; IG_{after,i} - IG_{before,i}.$$

Features for which ΔIG_i exceeds a positive threshold are treated as strengthened; features for which it falls below the symmetric negative threshold are treated as weakened. The new selected set is obtained from the current top-K ranking by adding strengthened candidates and removing weakened ones, with a length correction that brings the result back to size K if either operation overshoots.

For Avazu, Criteo, and the high-dimensional RBF stream, the selector keeps 20 features. For the Agrawal variants, where the feature space is small, it keeps 5 features. For the LED stream, it keeps 7 features, matching the number of truly relevant LED-segment attributes. Real CTR and RBF experiments use a 5,000-instance comparison window; Agrawal uses an 800-instance window; LED uses a 1,000-instance window. These values reflect the different dimensionality and drift dynamics of the streams.

The ADWIN input in this part of the pipeline is a single scalar error stream per evaluated model, not one ADWIN detector per feature. After each prediction, the evaluator sends $e_t = 0$ for a correct binary decision and $e_t = 1$ for an incorrect one. The ADWIN alarm therefore means that the model's recent error rate changed significantly. Feature-level changes are inferred after the alarm by comparing IG_{before} and IG_{after} on buffered instances.

Algorithm 2: Drift-aware feature selection after an ADWIN alarm

Input: Recent buffer B, top-K size K, post-drift window length W, threshold τ
Output: Updated selected feature set S_t, removed features R, added features A

if *ADWIN raises an alarm and no adaptation is already pending* **then**
  Freeze the current buffer as B_{before};
  Compute $IG_{before}(i)$ for every feature i on B_{before};
  Clear the rolling buffer and start collecting post-drift data;
if W *post-drift instances have been collected* **then**
  Let B_{after} be the collected post-drift buffer;
  Compute $IG_{after}(i)$ for every feature i on B_{after};
  Compute $\Delta IG_i = IG_{after}(i) - IG_{before}(i)$;
  $S_{base} \leftarrow$ top-K features ranked by IG_{after};
  Add features with $\Delta IG_i > \tau$ to S_{base};
  Remove features with $\Delta IG_i < -\tau$ from S_{base};
  If more than K features remain, keep the K highest by IG_{after};
  $R \leftarrow S_{old} \setminus S_{new}$ and $A \leftarrow S_{new} \setminus S_{old}$;
  Rebuild the filtered header and notify the model adaptation listener;

This update rule also prevents a feature with low absolute Information Gain from replacing a feature that remains highly informative merely because the low-IG feature is improving. The candidate set is anchored in the post-drift top-K ranking and, whenever the set grows beyond K, it is shrunk again according to IG_{after}. Therefore a low but growing feature can enter only when it has become competitive after the drift or when another selected feature has genuinely weakened.

Figure 9 summarises the control flow of the drift-aware selector as an event-driven, two-phase process.

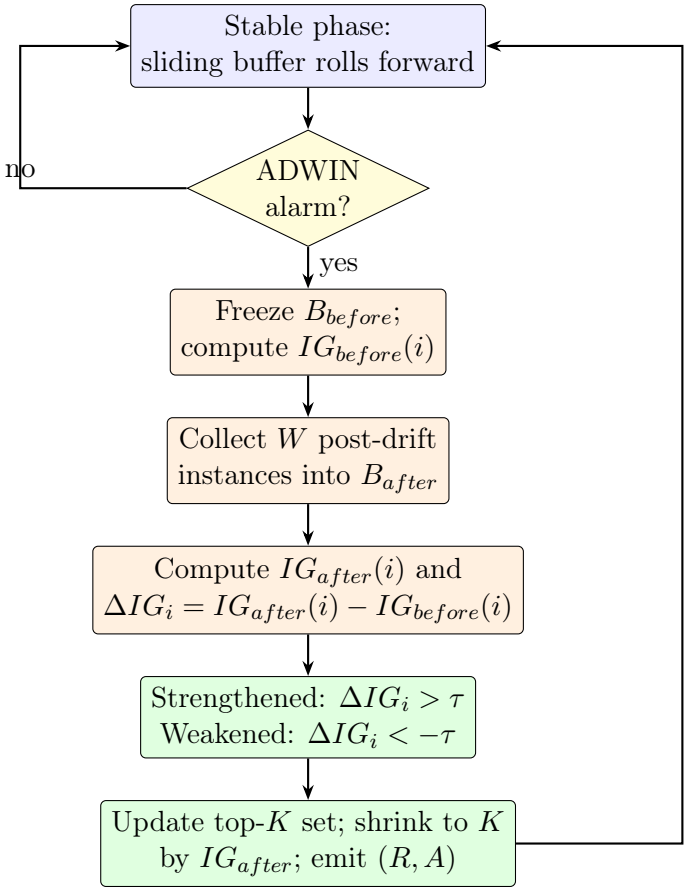


Figure 9: Control flow of the drift-aware feature selector (Algorithm 14). While stable, the buffer simply rolls forward; an ADWIN alarm freezes the pre-drift Information Gain, a further window of W instances is collected, and the selected set is updated from the signed Information Gain change ΔIG_i.

4.4 Drift-Aware SRP

The proposed model, Drift-Aware SRP (DASRP), extends the SRP framework along two complementary axes. First, ensemble subspaces become responsive to feature drift instead of being fixed at construction time. Second, members whose subspaces are most heavily affected by the drift do not vote at full strength immediately: the ensemble keeps track of how much fresh evidence each such member has seen and weighs its vote accordingly during recovery. Both mechanisms are designed to avoid the failure mode of a naive reset, in which the ensemble loses all accumulated knowledge on every drift event.

The implementation is split between two helper classes used by DASRP. The `SubspaceManager` owns the per-learner feature subsets and exposes an `adaptSubspaces` operation that consumes the weakened and strengthened feature sets produced by the drift-aware selector. For each learner, it counts the number of weakened features inside its subspace; learners below a configurable overlap threshold are left untouched, since their predictive basis was not affected by the drift. Learners above the threshold have their weak features replaced, preferring strengthened candidates and falling back to neutral features when the strengthened pool is too small. Every learner whose subspace is actually modified is then reinitialised on its new sub-header, since a Hoeffding tree built on the old attribute set cannot be reused consistently once that attribute set changes.

The reset ratio does *not* decide whether a learner is reinitialised; instead, the fraction of the original subspace that had to be replaced is compared against the reset ratio only to decide which of the reinitialised learners are additionally flagged for voting-weight reduction during recovery.

Formally, let S_m be the current subspace of learner m and W the weakened feature set emitted by the selector. The learner is adapted only if it shares at least ω weak features,

$$|S_m \cap W| \;\geq\; \omega \quad \text{(minimum overlap)},$$

and, once adapted, it is additionally flagged for weight reduction when the weakened fraction of its old subspace reaches the reset ratio ρ,

$$o_m \;=\; \frac{|S_m \cap W|}{|S_m|} \;\geq\; \rho.$$

In our experiments $\omega \in \{1, 2\}$ and $\rho = 0.5$.

The `WeightManager` mirrors this weight decision on the voting side. Members flagged by the subspace manager as exceeding the reset ratio have their weight reduced to a configurable value (0.3 in our experiments) and recover linearly toward the normal weight 1.0 at a fixed rate per training instance. Aggregation is performed as a weighted average,

$$\hat{p} \;=\; \frac{\sum_{i=1}^{N} w_i \, p_i}{\sum_{i=1}^{N} w_i},$$

so that freshly reset members influence the ensemble in proportion to the amount of new data they have processed. A flagged learner m has its weight set to w_{reset} at the reset instant t_m^0 and then recovers linearly,

$$w_m(t) \;=\; \min\!\big(w_{\text{norm}},\; w_{\text{reset}} + r\,(t - t_m^0)\big),$$

with $w_{\text{reset}} = 0.3$, $w_{\text{norm}} = 1.0$, and recovery rate $r = 0.001$ per instance. With a recovery rate of 0.001 per instance, full recovery takes roughly $\frac{w_{\text{norm}} - w_{\text{reset}}}{r} = 700$ instances, which is comparable to the data volume needed by a Hoeffding tree to perform its first split.

DASRP can remove a feature from a learner only when that feature appears in the selector's weakened set and the learner contains enough weakened features to pass the overlap threshold. Thus, potentially relevant features are not globally removed from the whole ensemble at once: unaffected learners keep their old subspaces and continue voting. The final experiments show that this selective repair is useful on high-dimensional CTR streams, but simple synthetic streams with frequent ADWIN alarms can still cause over-adaptation. For this reason, stricter thresholds and per-stream calibration are treated as part of the final recommendations rather than as a universal default.

The adaptation cycle ties everything together as follows:

1. ADWIN detects a change in the online error stream.
2. The drift-aware selector compares Information Gain before and after the drift and emits the weakened and strengthened feature sets.
3. The `SubspaceManager` removes weak features from affected subspaces and fills the gaps using strengthened features whenever possible.
4. Every learner whose subspace was modified is reinitialised on its new sub-header.

5. Learners whose replaced fraction exceeds the reset ratio are additionally flagged for weight reduction; the `WeightManager` lowers their vote weight and gradually restores it during the subsequent training instances.

Figure 10 shows how a single ADWIN alarm is turned into a per-learner repair-or-keep decision and the subsequent weighted vote.

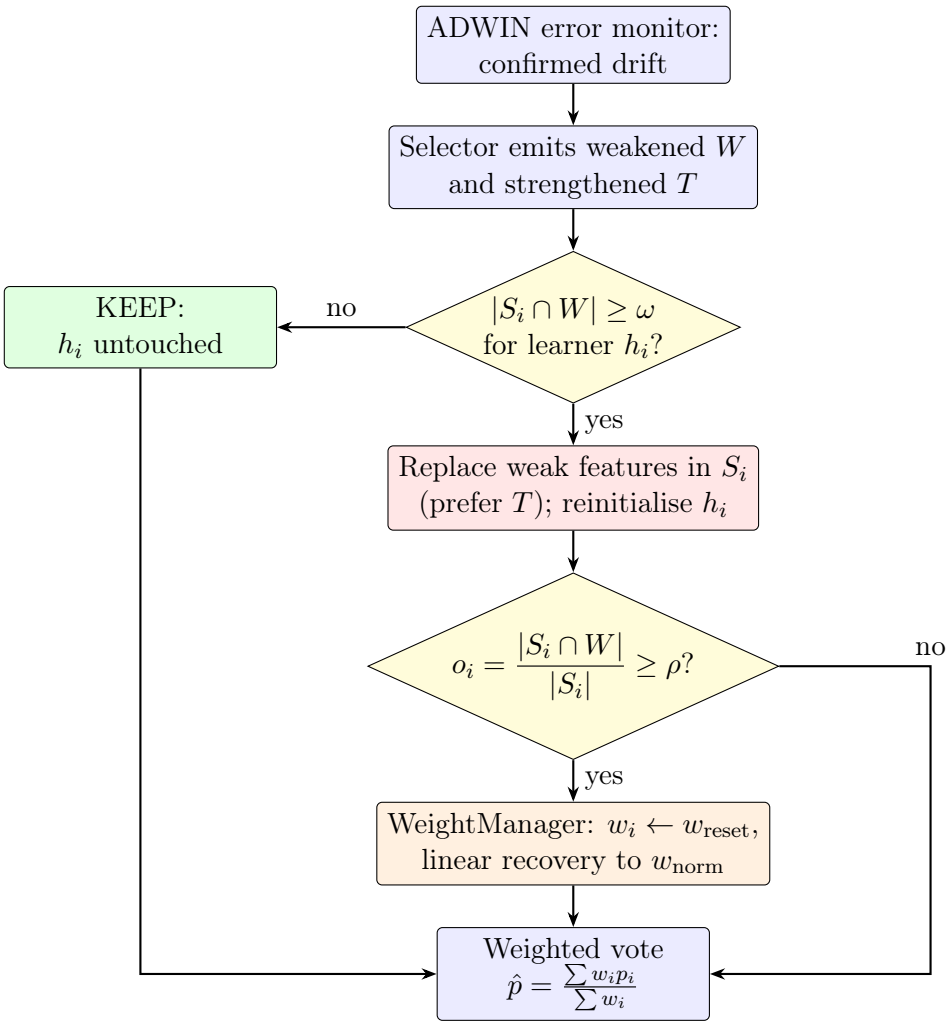


Figure 10: DASRP adaptation cycle. After a confirmed drift, each learner h_i is kept untouched when its subspace overlaps the weakened set by fewer than ω features; otherwise its weak features are replaced and the learner is reinitialised. Only learners whose weakened fraction o_i reaches the reset ratio ρ have their voting weight reduced and then linearly recovered. The ensemble prediction is the weight-normalised average of the member probabilities.

DASRP uses online bagging with $\lambda = 1$ rather than the $\lambda = 6$ value of the reference SRP implementation; this reduces the amplification of repeated training and keeps the focus of the experiments on the contribution of the drift-aware components. Drift detection is handled at the ensemble level by the external ADWIN monitor rather than by per-learner detectors, which keeps the adaptation policy concentrated in a single place.

On real CTR datasets, DASRP uses 10 base Hoeffding-tree learners, subspaces of size 20, ADWIN with $\delta = 0.002$, reset weight 0.3, recovery rate 0.001, and a reset ratio of 0.5.

On Agrawal variants, the configuration uses subspaces of size 4, ADWIN with $\delta = 0.001$, reset weight 0.5, and recovery rate 0.005. On LED, the subspace size is 7 so that a learner can in principle cover all relevant LED-segment attributes. On the high-dimensional RBF stream, the subspace size is 20, matching the real CTR setting. The different settings reflect the feature-space size and drift dynamics of each stream family.

5 Final evaluation

5.1 Experiment matrix

The final evaluation used the CSV outputs from the implemented Java experiment pipeline. The baseline matrix contains 63 runs: 7 datasets, 3 models (HT, HAT, SRP), and 3 feature selectors (none, static top-K, online ranking). The drift-aware matrix contains 21 additional runs: for each dataset, HT with the drift-aware selector, HAT with the drift-aware selector, and DASRP. In total, the report summarizes 84 prequential experiments. The seven datasets are Avazu, Criteo, Agrawal sudden, Agrawal gradual, Agrawal noisy, LED irrelevant-attribute drift, and high-dimensional RBF.

Each run processes 200,000 instances. Metrics are logged every 1,000 instances. The main metrics are final Accuracy, final Log Loss, final AUC, and windowed 1,000-instance Log Loss. Log Loss is the most important metric for CTR prediction because the output probability is used directly in ranking and bidding decisions. AUC is treated as a secondary metric. It is directly comparable for the binary CTR, Agrawal, and RBF streams. For the 10-class LED stream, the implementation effectively reports a class-1-versus-rest AUC, so LED results are interpreted primarily through Accuracy, Log Loss, and windowed Log Loss.

5.2 Real CTR results

Table 3: Final metrics on real CTR streams (Avazu, Criteo). The lower Log Loss, the better.

Dataset	Method	Accuracy	Log Loss	AUC
Avazu	HT + no selector	0.8052	1.0169	0.6339
Avazu	SRP + online ranking	0.8206	0.4889	0.6615
Avazu	SRP + static top-K	0.8243	0.4659	0.7022
Avazu	**DASRP + drift-aware SRP**	**0.8253**	**0.4332**	**0.7048**
Criteo	HT + no selector	0.7497	1.0110	0.6297
Criteo	SRP + online ranking	0.7557	0.5522	0.6512
Criteo	**DASRP + drift-aware SRP**	**0.7583**	**0.5291**	0.6511
Criteo	SRP + static top-K	**0.7634**	0.5622	**0.6754**

The most important result is that DASRP achieved the lowest final Log Loss on both real CTR datasets. On Avazu, DASRP improved Log Loss from 0.4659 for the best SRP baseline to 0.4332, which is a relative reduction of about 7.0%. On Criteo, DASRP improved Log Loss from 0.5522 to 0.5291, a relative reduction of about 4.2%. This

supports the hypothesis that drift-aware feature adaptation is useful in high-dimensional CTR streams.

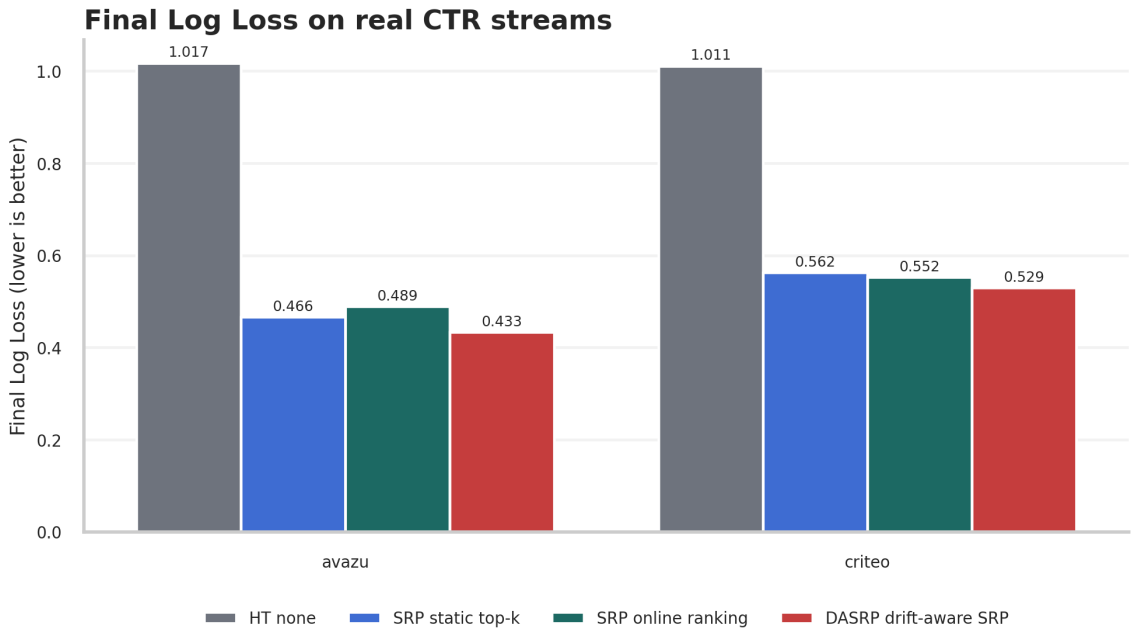


Figure 11: Final Log Loss comparison on Avazu and Criteo.

The time-series view in Figure 12 shows that the improvement is not only a final-point artifact. DASRP remains among the lowest-loss methods throughout the real CTR streams. The Avazu result is particularly stable. On Criteo, static SRP is competitive early but degrades later, while DASRP avoids the strongest late loss increase.

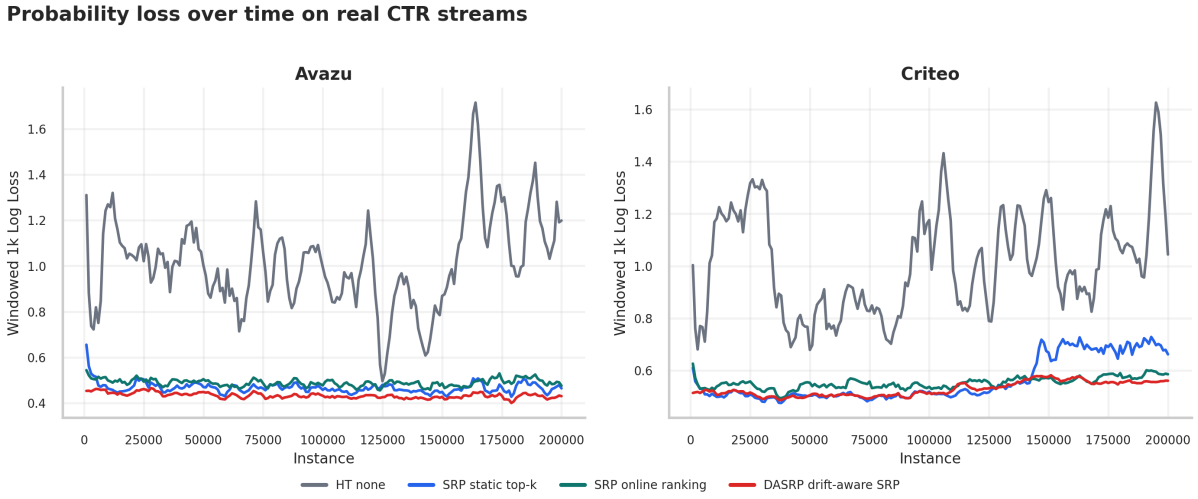


Figure 12: Left: Avazu dataset. Right: Criteo dataset. Windowed 1,000-instance Log Loss over time on real CTR streams:

5.3 Runtime analysis

Runtime is a key constraint in streaming systems. A model that provides good Log Loss but cannot process the stream fast enough is not practical. DASRP was much faster

than the strongest SRP baselines on real CTR streams because it uses a compact custom ensemble and, on drift, repairs only the subspaces of the affected learners, instead of running the heavier vanilla SRP baseline configuration with its per-learner background trees and detectors.

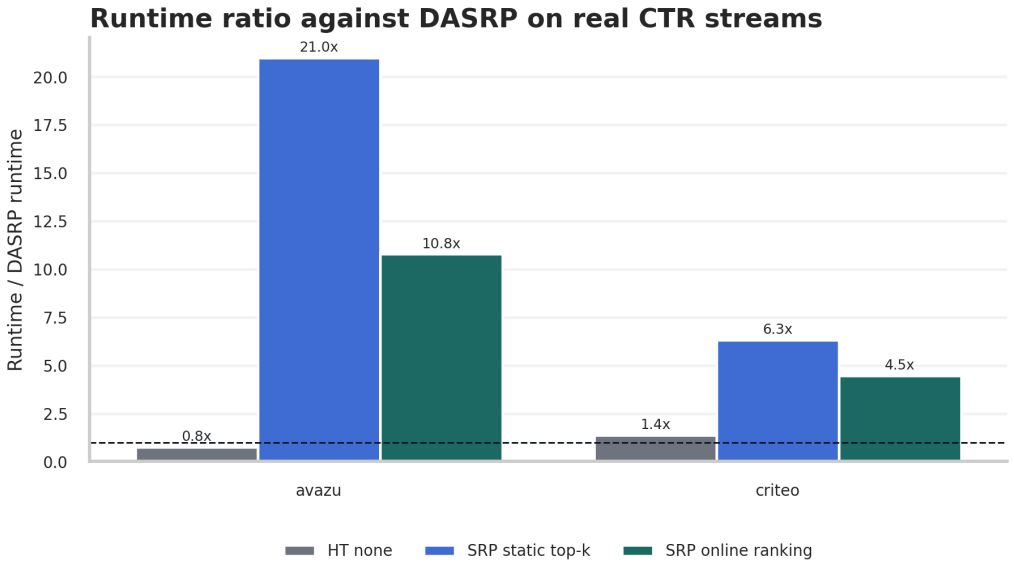


Figure 13: Avazu and Criteo datasets: Runtime ratio of selected real-stream baselines against DASRP. Values above 1 mean the baseline was slower than DASRP.

On Avazu, SRP with static top-K took approximately 21 times the DASRP runtime, and SRP with online ranking took approximately 10.8 times the DASRP runtime. HT was faster than DASRP, but it was much worse in final Log Loss. This indicates that DASRP is not the fastest method overall, but it provides an attractive quality/runtime trade-off among the competitive ensemble methods.

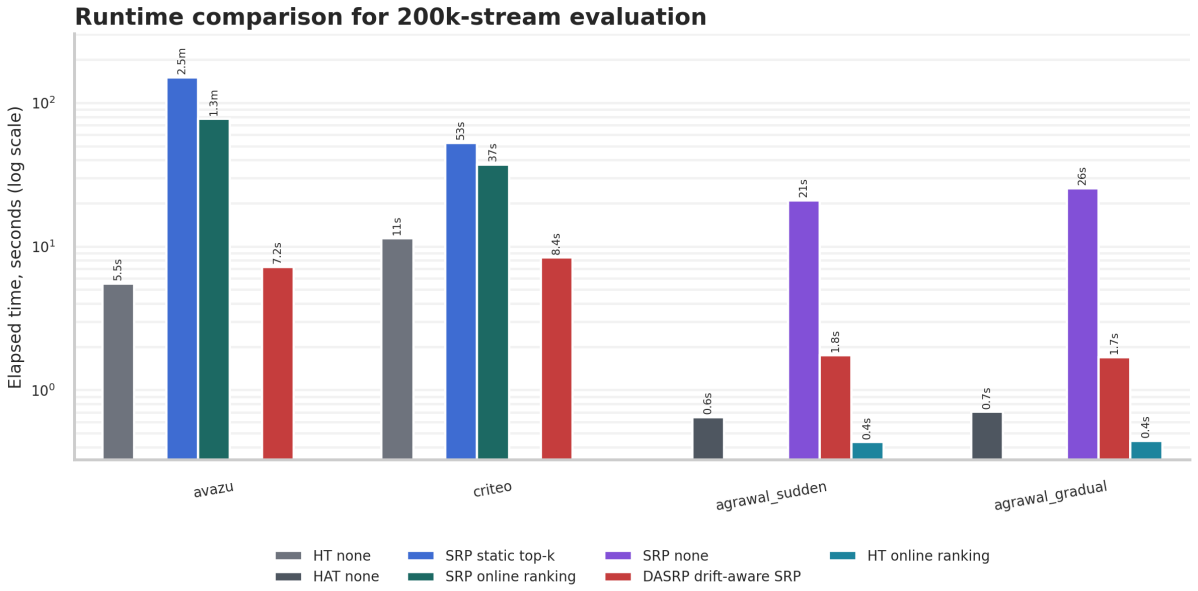


Figure 14: Avazu and Criteo datasets. Absolute runtime comparison for 200,000-instance runs. The vertical axis is logarithmic.

5.4 Synthetic-stream results

Table 4: Final metrics on controlled synthetic streams. For each stream, the strongest baseline by final Log Loss is compared with DASRP.

Dataset	Method	Accuracy	Log Loss	AUC
Sudden Agrawal	HAT + no selector	**0.9978**	**0.0118**	**1.0000**
Sudden Agrawal	DASRP + drift-aware SRP	0.8311	0.3809	0.8860
Gradual Agrawal	HAT + no selector	**0.9688**	**0.1123**	1.0000
Gradual Agrawal	DASRP + drift-aware SRP	0.9572	0.3191	**1.0000**
Noisy Agrawal	HAT + no selector	**0.9203**	**0.2651**	**0.9871**
Noisy Agrawal	DASRP + drift-aware SRP	0.7757	0.4869	0.8667
LED irrelevant	SRP + static top-K	0.1887	**3.0126**	0.7647
LED irrelevant	DASRP + drift-aware SRP	**0.1954**	3.2834	**0.9360**
RBF highdim	SRP + no selector	**0.9988**	**0.0084**	**1.0000**
RBF highdim	DASRP + drift-aware SRP	0.9250	0.2270	0.9672

The expanded synthetic evaluation satisfies the mandatory condition of testing reference and novel methods on several controlled streams. The results also reveal an important limitation: DASRP is not universally superior on simple synthetic generators. On Agrawal and RBF, the best reference baselines learn the generated concepts very quickly. DASRP is competitive on gradual Agrawal but underperforms on the noisier and high-dimensional RBF settings because frequent drift alarms cause too many feature/subspace repairs. On LED, where the relevant and irrelevant attributes are explicitly swapped, DASRP substantially reduces Log Loss compared with most tree baselines and is close to the best SRP top-K configuration.

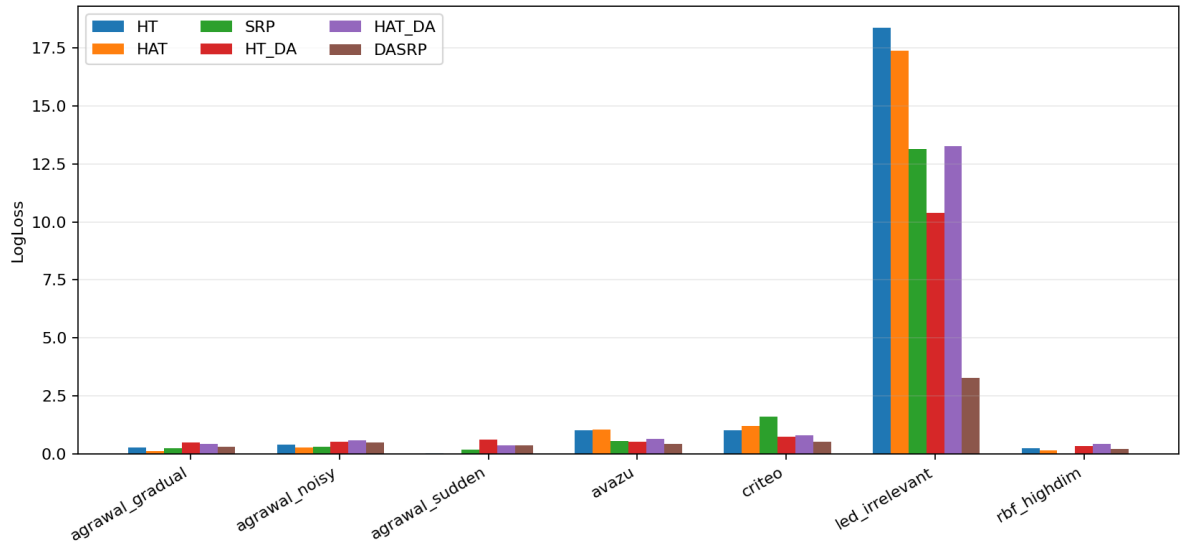


Figure 15: All seven evaluated datasets: final Log Loss for HT, HAT, vanilla SRP, HT with drift-aware selection, HAT with drift-aware selection, and DASRP. This plot explicitly includes vanilla SRP in the synthetic comparison.

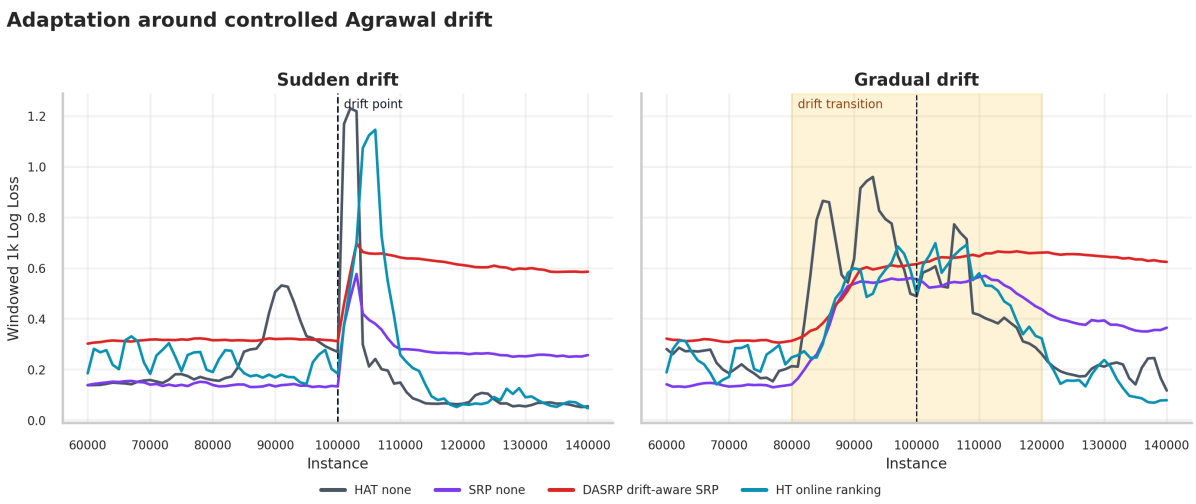


Figure 16: Agrawal sudden and gradual datasets: Windowed Log Loss around controlled Agrawal drift.

The number of ADWIN alarms also shows that drift-aware mechanisms require careful tuning. In simple synthetic streams, frequent alarms may lead to unnecessary feature and model changes.

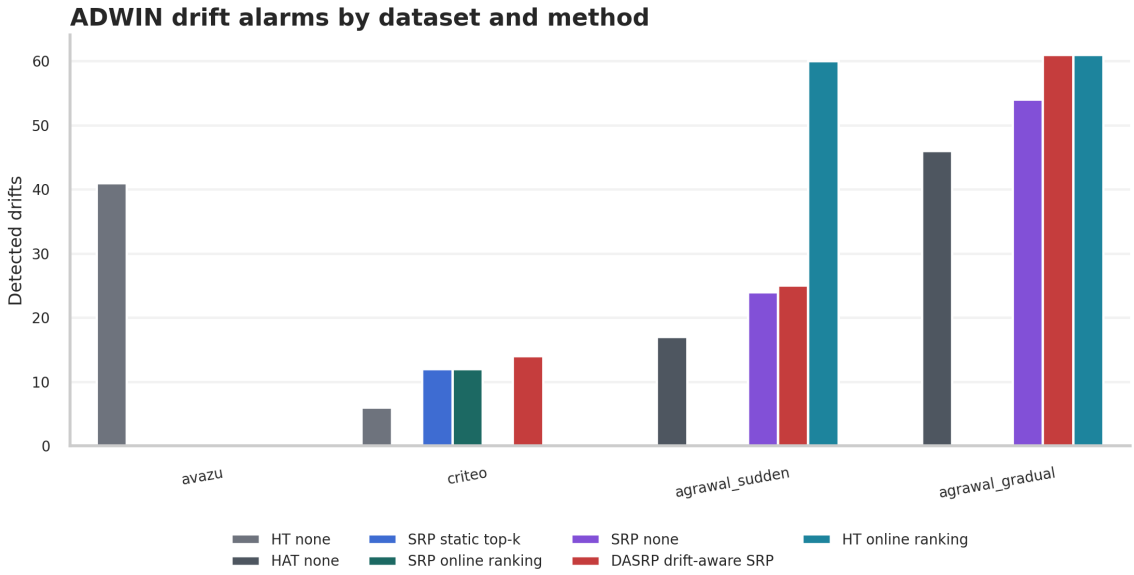


Figure 17: All evaluated datasets: ADWIN drift alarms by dataset and selected method.

6 Conclusion

The final experiments confirm that online feature adaptation can improve CTR prediction in high-dimensional, non-stationary streams. On Avazu and Criteo, DASRP achieved the best final Log Loss among the evaluated methods and was substantially faster than the strongest SRP baselines. This is the most important outcome of the project because CTR systems rely on calibrated probabilities rather than only hard class labels.

The expanded synthetic experiments provide an important limitation. On Agrawal and RBF streams, standard adaptive trees or vanilla SRP can recover very well because

the generated concepts are comparatively clean and quickly learnable. DASRP is therefore not universally superior. Its strongest use case is a stream where feature relevance itself is unstable and where selective subspace repair avoids the high cost of rebuilding a full ensemble. The LED irrelevant-attribute experiment supports this interpretation, while the noisy and high-dimensional synthetic runs show that ADWIN and IG thresholds require careful per-stream tuning.

Future work should include full million-instance experiments, a formal recovery-time metric around drift events, feature-level drift detectors in addition to the current error-rate ADWIN, and a more systematic search over ADWIN sensitivity, Information Gain thresholds, subspace sizes, reset ratio, and weight recovery rate.

7 Team contributions

Table 5: Division of work across team members, including detailed contributions and time assessment.

Name	Contributions	Workload
Hubert Jaczyński	**Analytics and final evaluation:** concept drift detection in raw data with ADWIN; final evaluation and comparative analysis of the proposed algorithm against baselines. **Report:** final evaluation and conclusion sections. **Presentation creation.** **Revision:** added Hoeffding Tree results to the real CTR table, added the SRP + online ranking row for Avazu, produced the model-comparison plot including vanilla SRP, extended the synthetic-stream results and drift-count analysis, and clarified the error-rate basis of the ADWIN alarms.	∼45h
Aleksandra Kłos	**Data engineering and baseline evaluation:** preprocessing Avazu and Criteo streams, exploratory data analysis, feature hashing, missing-value imputation, baseline evaluation in MOA, and scripts for Log Loss and adaptation-gap analysis. **Report:** introduction, data preparation, and evaluation methodology sections. **Presentation creation.** **Revision:** corrected the Criteo rolling-statistics computation so it is performed before zero-imputation and regenerated the ARFF file, added dataset names to all figure captions, fixed and described the `site_x_hour` feature, regenerated the HT adaptation-gap experiment at the consistent 200,000-instance scale, and added the comment on additional drift types in Subsection 3.1.	∼45h
Jakub Oganowski	**Algorithm implementation:** Java implementation of the SRP/OFS integration, drift-aware feature selection, subspace adaptation, and ensemble weighting. **Report:** architecture of the proposed solution section. **Presentation creation.** **Revision:** added the `algorithm2e` pseudocode for the main pipeline and the drift-aware selector, implemented the three new synthetic stream providers (noisy Agrawal, LED, high-dimensional RBF), clarified the single error-rate ADWIN input and the model-reset behaviour, and documented the safeguard against low Information Gain features by replacing high Information Gain ones.	∼45h

8 Updates compared to the preliminary report

All newly added or revised content is highlighted in blue throughout the report. Table 6 maps each reviewer remark to the section(s) modified.

Table 6: Changes addressed in response to the reviewer comments.

Reviewer remark	**Change applied**	**Changed section(s)**
Add pseudocode, e.g. with algorithm2e, for the main pipeline and drift-aware feature selection.	Added two `algorithm2e` blocks: the prequential evaluation/adaptation pipeline and the ADWIN-triggered drift-aware feature-selection procedure.	Subsection 4.1, Algorithm 15; Subsection 4.3, Algorithm 14.
Clarify how feature selection influences existing models such as HT, and whether past models are dropped or new splits are constrained.	Clarified that HT, HAT, and vanilla SRP receive physically filtered instances. When a dynamic selector changes the feature subset, the model is reinitialised on a new filtered header; old splits are dropped, not retroactively constrained. For DASRP, every ensemble learner whose subspace is modified is likewise reinitialised; the reset ratio only controls which of those learners also have their voting weight temporarily reduced.	Subsection 4.2, Subsection 4.3, and Subsection 4.4.
Clarify the input of ADWIN. Is there one ADWIN instance per feature?	Clarified that the Java evaluation uses one ADWIN detector per evaluated model run. Its input is the online 0/1 prediction-error stream. There is not one ADWIN instance per feature.	Subsection 4.3 and Algorithm 15.
Add dataset names to plot captions; fix `site_x_hour`; explain how IG is computed and based on what data.	Added dataset names to captions, fixed the `site_x_hour` formatting, defined it as a site/hour interaction, and described IG estimation on warm-up, sliding, pre-drift, and post-drift buffers.	Subsection 2.3, Subsection 3.1, Subsection 4.2, figure captions.
Reconsider Criteo rolling statistics because imputed values may disturb the calculations.	Clarified and revised the preprocessing description: rolling statistics for `I1`–`I5` are computed before zero-imputation, with missing values skipped by the window estimator. Remaining missing original values are then zero-filled.	Subsection 2.3.
Add at least three more synthetic streams, including one larger dimensionality stream.	Added `agrawal_noisy`, `led_irrelevant`, and `rbf_highdim`. The RBF stream has 100 attributes. The final evaluation now contains 63 baseline and 21 drift-aware runs over seven datasets.	Subsection 2.4, Table 2, Subsection 5.1, Subsection 5.4.
Comment whether drift detected in Subsection 4.2 is the only drift present; add drift detection experiments based on other features and error rates.	Added an explicit comment that CTR drift is not the only possible drift. The main Java experiments additionally log ADWIN alarms from model error-rate streams, and the final evaluation reports drift counts by dataset and method.	Subsection 3.1, Subsection 4.3, Fig. 17.
Use Hoeffding Tree as another baseline.	Added HT to the baseline model set and explicitly included HT rows in the real CTR result table. HT is also included in the committed 63-run baseline summary.	Subsection 4.2, Table 3, Subsection 5.1.
SRP + online ranking results are missing for Avazu.	Added the Avazu SRP + online ranking row to the real CTR result table and kept the corresponding row in the committed results CSV.	Table 3; `results_summary.csv`.

Could a feature of low yet growing IG replace a high-IG feature that is not improving? Consider modifying the mechanism.	Clarified the implemented safeguard: the selected set is anchored in the post-drift top-K ranking and is shrunk by IG_{after} if it grows beyond K. Thus a low-IG feature cannot replace a still-high-IG feature solely because its IG increased.	Subsection 4.3.
Fig. 11 should include vanilla SRP.	Added a final Log Loss comparison plot over all seven datasets that explicitly contains vanilla SRP together with HT, HAT, HT-DA, HAT-DA, and DASRP.	Fig. 15, Subsection 5.4.
DASRP may drop relevant features. Analyse which features are left and possibly revise DASRP.	Clarified that DASRP repairs only learners whose subspaces overlap sufficiently with weakened features. Unaffected learners preserve their old subspaces. The final discussion also identifies over-adaptation under frequent alarms as a limitation requiring per-stream threshold calibration.	Subsection 4.4, Subsection 5.4, Conclusion.

A Github repository link

Most of the data, code, results, and synthetic data stream configurations developed for this project are available in the following repository:

`https://github.com/Klossek02/MLDataStreams`

References

[1] H. Brendan McMahan et al. "Ad click prediction: a view from the trenches". In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2013, pp. 1222–1230.

[2] Pedro Domingos and Geoff Hulten. "Mining high-speed data streams". In: *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2000, pp. 71–80.

[3] Joao Gama et al. "A survey on concept drift adaptation". In: *ACM Computing Surveys* 46.4 (2014), pp. 1–37.

[4] Haiguang Li et al. "Group feature selection with streaming features". In: *2013 IEEE 13th International Conference on Data Mining*. 2013, pp. 1109–1114.

[5] Heitor Murilo Gomes, Jesse Read, and Albert Bifet. "Streaming random patches for evolving data stream classification". In: *2019 IEEE International Conference on Data Mining*. 2019, pp. 240–249.

[6] Congcong Liu et al. "On the adaptation to concept drift for CTR prediction". In: *arXiv preprint arXiv:2204.05101* (2022).

[7] Vladimir Vovk. "The fundamental nature of the log loss function". In: *Fields of Logic and Computation II*. Springer, 2015, pp. 307–318.

[8] Albert Bifet et al. “MOA: Massive online analysis, a framework for stream classification and clustering”. In: *Proceedings of the First Workshop on Applications of Pattern Analysis*. 2010, pp. 44–50.

[9] Kilian Weinberger et al. “Feature hashing for large scale multitask learning”. In: *Proceedings of the 26th Annual International Conference on Machine Learning*. 2009, pp. 1113–1120.

[10] Rakesh Agrawal, Tomasz Imielinski, and Arun Swami. “Database mining: A performance perspective”. In: *IEEE Transactions on Knowledge and Data Engineering* 5.6 (1993), pp. 914–925.

[11] Albert Bifet and Ricard Gavalda. “Learning from time-changing data with adaptive windowing”. In: *Proceedings of the 2007 SIAM International Conference on Data Mining*. 2007, pp. 443–448.

[12] Albert Bifet and Ricard Gavalda. “Adaptive learning from evolving data streams”. In: *International Symposium on Intelligent Data Analysis*. 2009, pp. 249–260.

[13] Tom Fawcett. “An introduction to ROC analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.

[14] Geoff Hulten, Laurie Spencer, and Pedro Domingos. “Mining time-changing data streams”. In: *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2001, pp. 97–106.

List of Figures

List of Tables